# Verification & Validation

*Are we building the product right — and are we building the right product?*

UNIVERSITY QUESTIONS · SIX APPEARANCES COMBINED

*"Discuss V&V activities" — May 16, May 18, Dec 19 · "Differentiate verification and validation" — Dec 17, Dec 19 · "Discuss the benefits of V&V in a project" — May 17*

· TWO WORDS, TWO QUESTIONS

***Verification asks: "Are we building the product RIGHT?"***
***Validation asks: "Are we building the RIGHT product?"***

— the classic formulation popularized by Barry Boehm

Same letters, same industry, constantly confused — yet they are different activities, done by different people, using different methods, at different times. Getting the distinction crisp is worth marks in three separate University Questions.

(I)

# Verification: The Static Inspection

Verification is the process of checking documents, design, code, and program — to confirm the software is being built according to the requirements. Its main goal: ensure the quality of the application, its design, its architecture. Crucially, the code is NOT executed — everything is examined at rest, on paper and on screen.

*Performed by: the QA team, checking the software against the SRS document.*

## THE VERIFICATION TOOLKIT

— **Reviews**

*A document is examined formally by peers or seniors, defects logged.*

— **Walkthroughs**

*The author leads colleagues through the work, step by step.*

— **Inspections**

*The most formal: trained moderator, defined roles, exit criteria.*

— **Desk-checking**

*One person traces the logic by hand, alone at the desk.*

(II)

# Validation: The Dynamic Trial

Validation is a dynamic mechanism — testing and validating whether the actual software product meets the exact needs of the customer. It ensures the software fulfils its desired use in an appropriate environment. The code is ALWAYS executed: the product runs, and reality gets a vote.

1
**Unit Testing**
Each smallest piece, alone

→

2
**Integration Testing**
Pieces combined, interfaces checked

→

3
**System Testing**
The whole product, end to end

→

4
**User Acceptance Testing**
The customer's own hands

*Also in the toolkit: Black Box Testing, White Box Testing, and non-functional testing — all validation methods, because all of them run the code.*

# Verification vs. Validation — Eight Differences

| # | Verification | Validation |
|---|---|---|
| 1 | Checks documents, design, code, and program | A dynamic mechanism — tests the actual product |
| 2 | Does NOT involve executing the code | ALWAYS involves executing the code |
| 3 | Reviews, walkthroughs, inspections, desk-checking | Black box, white box, non-functional testing |
| 4 | Checks conformance to the SPECIFICATION | Checks the customer's requirements & EXPECTATIONS |
| 5 | Finds bugs EARLY in the development cycle | Finds bugs verification CANNOT catch |
| 6 | Targets architecture, design, database, specs | Targets the actual product |
| 7 | Done by the QA team, against the SRS | Testing team, with QA, executes on the code |
| 8 | Comes BEFORE validation | Comes AFTER verification |

# Why a Project Needs Both

*The benefits of V&V — assembled from the eight differences you just learned.*

B1

## Bugs caught early are bugs caught cheap

Verification finds defects in documents and design — before a line of faulty code is ever written, when fixes cost the least.

B2

## Two nets catch what one net misses

Validation finds the bugs verification cannot — runtime behaviour, integration failures, environment surprises.

B3

## Conformance AND satisfaction

Verification proves the product matches its specification; validation proves the specification matched the customer. Quality needs both.

B4

## Clear ownership, clear sequence

QA verifies against the SRS; the testing team validates the running code — verification first, validation after. Roles and order prevent gaps.

ENGINEERING CASE STUDY

# Mars Climate Orbiter, 1999

*Every component verified against its own logic — and a $327 million spacecraft lost to a unit conversion.*

NASA's orbiter flew 669 million kilometres to Mars, guided by two pieces of software: ground software built by Lockheed Martin that calculated thruster firings, and NASA's navigation software that consumed those numbers. NASA's interface specification required the data in metric units — newton-seconds. Lockheed's software produced it in imperial units — pound-force seconds. Every number passed between them was silently wrong by a factor of 4.45.

## CLASSIFY IT

*Each program 'worked' by its own internal logic — but was never verified against the interface specification, and the combined system was never validated end to end.*

# 4.45×

*the silent error in every thruster calculation — the ratio between pound-force and newtons*

| MISSION COST LOST | ALTITUDE: PLAN VS REAL |
| --- | --- |
| **$327M** | **226 → 57 km** |

*On September 23, 1999, the orbiter entered Mars' atmosphere at 57 km — far below the ~80 km survival threshold — and was destroyed. Nine months of accumulating error, discovered in one morning.*

ROOT CAUSE

## Verified in isolation; never validated together

The interface specification — the SIS document — clearly said metric. Verification against that document would have caught the mismatch on paper, before launch. And during the cruise, navigators noticed the trajectory drifting and raised informal concerns — but the anomaly was never escalated through the formal reporting process, and no end-to-end validation of the thruster data chain was performed.

*NASA's review board listed the units error as the cause — and the failure to catch it through verification and validation processes as the deeper failure.*

WHAT THIS CASE TEACHES

# Four Transferable Lessons

LESSON 01

## ‘Works by itself’ proves almost nothing

Both programs ran flawlessly by their own logic. Correctness is defined against the specification — not against the code's own assumptions.

LESSON 02

## Interfaces are where verification earns its keep

A desk-check of the outputs against the SIS document — static, cheap, pre-launch — would have exposed the units mismatch.

LESSON 03

## Validation is reality's vote — schedule it

No end-to-end trial of the full data chain was ever run. What isn't validated is merely believed.

LESSON 04

## An informal concern is not a report

Navigators felt something was wrong for months. Suspicions that never enter the formal process cannot save a mission.

## · FROM PRINCIPLE TO MACHINERY

***A verification requirement provides the rules of verification for a piece of data — a verifiable data item.***

In enterprise systems, verification stops being a meeting and becomes an engine: rules attached to data, applied automatically at runtime. The rules include WHERE and HOW they apply — for instance, whether the engine verifies a data item once at the participant level, or separately for each specific case.

*Running example: DATE OF BIRTH. One organization verifies it once, centrally. Another requires it re-verified for every program the citizen applies to.*

· PROPERTIES (1–4)

# The Rulebook, Part One

## P1 · DUE DATE & WARNING DATE

Verification must be completed within a set number of days after an event — counted from case creation or from when evidence arrived. Warning days give the caseworker advance notice; no warning set means no warning given. Deadlines run on workflow machinery.

## P2 · LEVEL

Evidence counts only at the required strength. If the rule demands a level-5 item — an original birth certificate — a level-1 photocopy does not satisfy it. A combination of items forming a level-5 group can.

## P3 · FROM & TO DATES

The requirement is effective only for a period — and interacts with the effective dates of evidence. Income effective January–July? A payslip verifies it. July–December? A tax return. The active date of the evidence decides which item is needed.

## P4 · MINIMUM ITEMS

At least N items must be provided — e.g., minimum 2. A complete verification group counts as a single item; items and groups can combine to reach the minimum.

· PROPERTIES (5–7)

# The Rulebook, Part Two

## P5 · MANDATORY

If mandatory, the evidence and its cases cannot be ACTIVATED until the verification rules are met. If optional, evidence can go active even before it's verified.

## P6 · CLIENT-SUPPLIED

Flags whether it's the participant's job to supply the items — so a client is never asked for a document that should be sourced elsewhere. Informational only: no system processing attached.

## P7 · RE-VERIFICATION

What happens when a caseworker changes ACTIVE evidence? Three settings (not applicable to participant evidence):

### — RECERTIFY ALWAYS

Nothing carries over — the new In-Edit record must be recertified from scratch.

### — RECERTIFY IF CHANGED

Value (and dependents) unchanged? Verification is copied over. Changed? Nothing is copied.

### — NEVER RECERTIFY

Verification information is always copied to the In-Edit record, change or no change.

# Key Takeaways

01 Verification is static — documents, design, code checked without execution, via reviews, walkthroughs, inspections, desk-checking. ‘Building the product right.’

02 Validation is dynamic — the code always runs: unit → integration → system → UAT. ‘Building the right product.’

03 The exam table has eight rows — methods, execution, target, timing, ownership, and what each catches that the other cannot.

04 Mars Climate Orbiter: components verified in isolation, interface never checked, system never validated — $327M lost to a 4.45× units error.

05 Verification requirements turn the principle into machinery: due dates, levels, effective dates, minimum items, mandatory flags, and three re-verification modes.